Critical Reproduction Study — Machine-Learning Intrusion Detection on NSL-KDD

Course: Data Science in Cyber — Dr. Uri Itai · **Topic:** Intrusion Detection Systems (IDS)
Author: *(student)* · **Repository:** https://github.com/yosefshanaa/DataCyber

---

0. Abstract

A very large family of online tutorials and GitHub repositories report that machine-learning classifiers detect network intrusions on the **NSL-KDD** benchmark with **~99% accuracy**. We critically evaluate this claim. Holding the models, features and preprocessing **fixed** and varying **only the evaluation protocol**, we reproduce ~0.999 accuracy on a random split of `KDDTrain+` (confirmed by 5-fold cross-validation, 0.9989 ± 0.0002) and then show the **same Random Forest collapses to 0.78 accuracy on the official `KDDTest+`** — a **21.9-percentage-point** drop. Under imbalance-robust metrics the picture is worse: a trivial majority predictor already scores 0.569 accuracy, and the models **miss 95% of R2L and U2R attacks** — precisely the credential-theft and privilege-escalation intrusions that matter most. We trace the collapse to a deliberate **train→test distribution shift** (R2L prevalence rises 0.79% → 12.8%) and to the use of **Accuracy on imbalanced data**, and confirm the gap is **21.9 ± 0.0 pp across five random seeds** (not a lucky split). Crucially, we **test the implied fix** rather than merely asserting it: a one-class anomaly detector trained on *normal traffic only* — a paradigm the tutorials never try — recovers the rare attacks (R2L 0.05→0.47, U2R 0.15→0.78) and **outperforms every supervised model on the official test set** (MCC 0.72 vs 0.66; F2 0.84 vs 0.72), at the cost of a higher false-alarm rate. **Verdict: the ~99% claim is computed correctly but is not a valid measure of intrusion-detection capability — and a better paradigm exists for the attacks that matter.**

---

1. Summary of the Source

The problem being addressed. Network Intrusion Detection: given per-connection traffic features, classify each connection as *normal* or as an *attack* (optionally by family: DoS, Probe, R2L, U2R).

The source under review. The dominant NSL-KDD tutorial pattern, instantiated by popular public repositories — primarily *Network Intrusion Detection Using Machine Learning* (`github.com/abhinav-bhardwaj/Network-Intrusion-Detection-Using-Machine-Learning`) and the equivalent `github.com/Mamcose/NSL-KDD-Network-Intrusion-Detection`. These projects load `KDDTrain+`, encode features, perform a **random** `train_test_split`, train several classifiers (Random Forest, KNN, SVM, neural networks), and report **accuracy in the 98–99.9% range**. The shared, headline claim we test is: "*ML classifiers detect NSL-KDD intrusions with ~99% accuracy.*"

Why the problem is important. Intrusion detection is a frontline defence. As the course notes, "*a False Negative endangers the organisation, a False Positive exhausts it*" — so the

quality of the evaluation is as important as the model. A method advertised at 99% that silently misses the dangerous attacks gives defenders false confidence.

The proposed solution. Supervised classification on the 41 NSL-KDD features using off-the-shelf estimators, evaluated by Accuracy on a held-out random split.

The dataset used. NSL-KDD (Canadian Institute for Cybersecurity; Tavallaee et al., 2009), the de-duplicated successor to KDD'99. `KDDTrain+` = 125,973 records, `KDDTest+` = 22,544 records, 41 features (3 nominal: `protocol_type`, `service`, `flag`; 6 binary flags; 32 numeric counts/rates), plus a fine-grained attack `label` and an NSL-KDD `difficulty` score. Attacks group into four families: **DoS, Probe, R2L** (remote-to-local) and **U2R** (user-to-root).

The model/methodology employed by us. Faithful reproduction (including a **literal reconstruction of the tutorials' own preprocessing**, leak and all) + a **controlled A/B protocol experiment** with **multi-seed error bars**, robust EDA, leakage-safe feature engineering (scikit-learn `Pipeline` / `ColumnTransformer`), four supervised models (majority **baseline**, Logistic Regression, Random Forest, Gradient Boosting) **plus two semi-supervised anomaly detectors** (Isolation Forest, One-Class SVM) that test our own recommendation, feature-engineering ablations, an imbalance-aware metric suite, and error analysis. Everything is reproducible (`RANDOM_STATE = 42`).

---

2. Critical Evaluation of the Author's Claims

This is the heart of the project. We enumerate the claims, test each with our own evidence, and deliver a verdict.

2.1 Claims table

#	Claim (as made by the source pattern)	Evidence they provide	Our re-test	Result
C1	"ML detects NSL-KDD intrusions at ~99% accuracy."	Accuracy on a **random split** of `KDDTrain+`.	Reproduced (RF 0.9990; CV 0.9989) **and** re-ran on official `KDDTest+`.	**Refuted as a capability claim** — drops to 0.78 (−21.9 pp).
C2	Accuracy is an adequate success metric.	Single Accuracy number.	Computed MCC, Balanced Acc, PR-AUC, per-class recall; compared to a constant predictor (0.569).	**Refuted** — Accuracy hides the failure.
C3	The model "detects attacks."	Aggregate score.	Per-class recall on `KDDTest+`.	**Refuted for R2L/U2R** — recall 0.050 / 0.045.
C4	The held-out split fairly estimates generalisation.	Random split / k-fold.	Quantified train→test class shift; CV also ~0.99.	**Refuted** — the proper test set is a different distribution.
C5	The full feature set is informative.	Uses all 41 features.	Variance / correlation / leakage audit.	**Partially** — 1 constant feature, 9 redundant pairs, 1 leaky metadata column.

2.2 Are the claims supported by the evidence? (the controlled experiment)

We changed **only** the test data. Same features, same preprocessing, same seeds, same models.

Protocol A — random 80/20 split of `KDDTrain+` (what the tutorials do):

Model	Accuracy	MCC	ROC-AUC	PR-AUC
Baseline (majority)	0.5346	0.000	0.500	0.465
Logistic Regression	0.9839	0.9676	0.9978	0.9980
Random Forest	**0.9990**	**0.9981**	1.0000	1.0000
Gradient Boosting	0.9994	0.9987	1.0000	1.0000

5-fold stratified CV on `KDDTrain+` (Random Forest) = **0.9989 ± 0.0002**, so the high number is *not* a lucky split.

Protocol B — official `KDDTest+` (the correct evaluation):

Model	Accuracy	Balanced Acc	Precision	Recall	F1	F2	MCC	ROC-AUC	PR-AUC
Baseline (majority)	0.4308	0.500	0.000	0.000	0.000	0.000	0.000	0.500	0.569
Logistic Regression	0.7446	0.7664	0.9134	0.6091	0.7308	0.6525	0.5436	0.7935	0.8579
Random Forest	**0.7798**	0.8033	0.9689	0.6335	0.7661	0.6806	**0.6214**	0.9614	0.9640
Gradient Boosting	0.8032	0.8237	0.9693	0.6757	0.7963	0.7193	0.6552	0.9610	0.9662

The same Random Forest loses 21.9 accuracy points (0.9990 → 0.7798) when evaluated on the data the dataset designers actually intended for testing. The high precision (0.97) with mediocre recall (0.63) shows the model is conservative: when it flags an attack it is usually right, but it lets a **third of all attacks through**.

The gap is not a lucky split. Two independent robustness checks confirm it: (i) 5-fold stratified CV *within* `KDDTrain+` also returns **0.9989 ± 0.0002**, and (ii) repeating Protocol A across **five random seeds** gives Protocol-A accuracy **0.9989 ± 0.0001** against the fixed Protocol-B 0.7798, so the A→B drop is **21.91 ± 0.01 pp** — essentially seed-invariant. The collapse is a property of the *distribution shift*, not of any one split.

It is also not an artefact of our pipeline. To rule out the possibility that *our* leakage-safe preprocessing manufactured the effect, we reconstructed the tutorials' **own** recipe — `LabelEncoder` (ordinal)-coded nominal columns, raw unscaled values, and the leaky `difficulty` column **kept in** — and ran the same Random Forest. It reproduces **0.9996** on a random split and still **collapses to 0.8224 on `KDDTest+`** (−17.7 pp). (Note the leak makes both numbers look *better*, which only deepens the over-optimism.) The inflation is therefore

intrinsic to *evaluating in-distribution*, by either pipeline. See §4 for the matching audit of the original repository.

2.3 Is the evaluation methodology appropriate? (No — three failures)

1. **Wrong test distribution.** `KDDTest+` is intentionally *not* a random sample of `KDDTrain+`; it over-represents hard attacks to test generalisation (§3 of Tavallaee et al.). A random split discards this and measures *interpolation within one distribution*, not detection of novel attacks. This is a concept-drift analogue (cf. the course's SolarWinds / COVID drift examples).
2. **Accuracy on imbalanced data.** The course warns explicitly that "*if 99.9% of the system is normal, a model that always says 'normal' gets high Accuracy yet is worthless.*" Here a constant predictor already reaches **0.569** on `KDDTest+`; the model's 0.78 is far less impressive than "99%" suggested, and the gain over the trivial baseline is modest in MCC terms.
3. **No per-class reporting and no baseline.** Aggregate Accuracy masks that R2L/U2R recall is ~5%.

2.4 Weaknesses, limitations, and whether the conclusions are justified

- **NSL-KDD itself is dated** (derived from 1999 traffic); none of these models would face modern encrypted/zero-day traffic. A 99% claim implies a solved problem; it is not.
- **No temporal validity** exists in the data (no timestamps), so any "real-time IDS" framing built on it is unsupported.
- The source's conclusion ("ML solves NSL-KDD intrusion detection") is **not justified**. A more defensible conclusion: *ML separates DoS/Probe from normal well, but content-based remote attacks (R2L/U2R) remain largely undetected and require different methods.*

Where we contradict the source, the reason is explicit: the gap is fully explained by the distribution shift quantified in §3 below and by the metric substitution in §2.3 — both reproduced in the notebook.

---

3. Feature Engineering Analysis

Was feature engineering performed (by us)? Yes — as leakage-safe scikit-learn transformers fit on training data only. The 41 raw features become **121 model inputs** after encoding.

Transformations applied, and why:

Transform	Applied to	Why / problem addressed	Effect / evidence
Drop constant	`num_outbound_cmds`	Zero variance across all 125,973 rows → no information; dilutes feature importance (redundancy).	Removed 1 feature with zero cost.

Transform	Applied to	Why / problem addressed	Effect / evidence
`log1p`	16 heavy-tailed counts/bytes	Extreme right-skew (`src_bytes` skew = **190.7**, range 0–1.3e9) breaks scale-sensitive models and Z-score.	Skew of `src_bytes` falls from 190.7 to ≈1.6 (see Fig. 2); Z-score outlier count becomes meaningful.
StandardScaler	all numeric	Put scale-sensitive models (LogReg/SVM/KNN) on equal footing.	Required for LogReg convergence; harmless to trees.
One-Hot (`handle_unknown='ignore'`)	`protocol_type` , `service` , `flag`	No natural order among tcp/udp/icmp; `service` has 70 categories; test may contain unseen services.	Avoids the fake ordering a label-encoder injects; robust to unseen categories.

Is there redundancy in the system? How to spot and tackle it. Yes. Using **Spearman** rank correlation (chosen because the features are heavy-tailed and non-normal — Pearson would be distorted by the very outliers we documented), we found **9 feature pairs with |ρ| ≥ 0.9**, e.g. `serror_rate ↔ srv_serror_rate` (0.973), `rerror_rate ↔ srv_rerror_rate` (0.966), `same_srv_rate ↔ diff_srv_rate` (−0.920). **Statistical vs practical significance:** with n = 125,973 *every* correlation is "statistically significant" (p ≈ 0), so a p-value is useless for selection here — we therefore use a **practical** effect-size threshold (|ρ| ≥ 0.9) to flag redundancy that is large enough to matter. **How to spot:** rank-correlation matrix (Fig. 3), Variance Inflation Factor, mutual information, and exact-duplicate column checks. **How to tackle:** drop or merge collinear features, or compress with PCA. As the course notes, redundancy *splits feature importance across collinear columns and harms explainability* — a real concern for an IDS that analysts must trust.

Why Spearman and not Pearson — demonstrated, not asserted. We computed both coefficients on every numeric pair. They disagree most on exactly the heavy-tailed count features, and the decisive example is `num_compromised ↔ num_root`: **Pearson = 0.999** but **Spearman = 0.165** (the largest gap, 0.83). A Pearson-driven redundancy filter would *delete one of these as* "*redundant*" *and destroy real signal*, because a handful of rows with simultaneously huge values dominate the linear fit, whereas the ranks reveal the two features are barely monotonically related. Conversely `same_srv_rate ↔ diff_srv_rate` is weak under Pearson (−0.38) but strong under Spearman (−0.92). This is the concrete, data-driven justification for using the rank-based measure on this telemetry.

We acted on the redundancy, not just flagged it (ablation). Dropping one feature from each |ρ| ≥ 0.9 pair (6 columns) and re-evaluating the same Random Forest on `KDDTest+` leaves performance essentially unchanged while shrinking the model input from 121 to 115 dimensions — confirming the dropped columns carried no *unique* signal. Adding the three domain features below gives a small but **real, tested** gain (not an assumed one):

RF on KDDTest+	Model inputs	Accuracy	MCC	Recall (attack)	F2
Full (41 features)	121	0.7798	0.6214	0.6335	0.6806
Pruned (−6 redundant)	115	0.7731	0.6120	0.6212	0.6693
+ Engineered (3 created)	124	**0.7863**	**0.6310**	**0.6450**	**0.6912**

A leakage trap we caught. The `difficulty` column is **metadata** (how many of 21 baseline learners classified the row correctly), not a runtime feature, and it differs by class (mean 20.32 for normal vs 18.57 for attack). Including it — as some tutorials do — is **target leakage**. We exclude it.

Was the feature engineering meaningful? (math + cyber). Yes. *Mathematically*, `log1p` linearises multiplicative byte/count scales and tames the tail that inflates the mean and standard deviation; one-hot avoids imposing a false metric on nominal protocols. *In cyber terms*, the engineered view preserves the signals that separate attack families — e.g. the `protocol × family` crosstab shows DoS concentrates on `icmp` / `tcp` while R2L rides `tcp` — so the encoding lets the model exploit protocol-specific structure instead of erasing it.

Additional features that could help. (1) Session-level **Δt** / inter-arrival aggregates if raw logs were available; (2) byte-ratio / asymmetry features (exfiltration signal) — we provide `total_bytes`, `bytes_ratio`, `error_rate_mean` in `src/features.py`; (3) frequency/target encoding of `service` to cut dimensionality; (4) entropy of destination ports per host (scan signal).

---

4. Reproducibility Analysis

- **Does the code run?** Our pipeline runs end-to-end from a clean environment (`requirements.txt`, `RANDOM_STATE=42`, `Restart & Run All`). The original tutorials are typically runnable but **pinned to old library versions** and frequently depend on a pre-cleaned CSV rather than the raw NSL-KDD files.
- **We audited the actual source repository** (`abhinav-bhardwaj/...`), not just the pattern. Its notebooks confirm, in code: a random `train_test_split`, `LabelEncoder` on the nominal columns, the `difficulty` / `level` column **retained as a feature**, and reported accuracies of **KNN ≈ 98.5%** and a **neural net ≈ 97.8%**. Decisively, the repository **never loads `KDDTest+` as a held-out distribution** — every headline number is computed on an in-distribution random split. This is the direct, repository-level confirmation of our central critique. Our literal reconstruction of that recipe (§2.2) reproduces 0.9996 on a random split and collapses to 0.8224 on `KDDTest+`.
- **Are files/dependencies available?** The dataset is public; we fetch it deterministically (`data/get_data.py`). Several source repos commit a derived CSV with **undocumented preprocessing** (re-encoded categoricals, dropped columns) — a hidden step that obscures what was actually done.
- **Hidden preprocessing.** The most consequential "hidden" choice is *implicit*: using a random split instead of `KDDTest+`. It is rarely stated as a limitation, yet it determines the headline result. A second, subtler one is keeping `difficulty` — a leak that further inflates the score.
- **Overall reproducibility verdict.** The *numbers* are reproducible; the *claim* is not robust. A result that reproduces but does not generalise is a reproducibility success and

a **validity failure** — exactly the distinction this project is meant to surface.

---

5. Experimental Results

Experiments performed. (a) Reproduce Protocol A; (b) 5-fold CV; (c) evaluate Protocol B on `KDDTest+`; (d) multiclass (5-class) evaluation; (e) per-class recall; (f) threshold sweep; (g) **literal tutorial-recipe reproduction**; (h) **multi-seed A→B stability**; (i) **feature-redundancy and engineered-feature ablations** (§3); (j) **anomaly-detection paradigm** (Isolation Forest, One-Class SVM).

Modifications we introduced. A trivial **baseline**, the **A/B protocol** control with **error bars**, the **imbalance-robust metric suite**, **per-class** error analysis, **feature ablations**, and — most importantly — a **semi-supervised anomaly-detection paradigm** that tests our own recommendation. None of these appear in the source.

Metrics used — definition and cyber interpretation. (TP/TN/FP/FN = true/false positives/negatives, attack = positive.)

Metric	Mathematical definition	Cybersecurity interpretation
Accuracy	(TP+TN)/(TP+TN+FP+FN)	Overall correctness; **misleading** under imbalance — reported only as a foil.
Precision	TP/(TP+FP)	Of raised alerts, the fraction that are real attacks; low → analyst alert fatigue.
Recall (TPR)	TP/(TP+FN)	Of real attacks, the fraction caught; low → **missed intrusions** (the costly error).
F1	2·P·R/(P+R)	Harmonic mean of precision & recall (single balance score).
Fβ (β=2)	$(1+\beta^2)\cdot P\cdot R/(\beta^2\cdot P+R)$	Recall-weighted F-score — encodes that a miss (FN) is worse than a false alarm (FP).
Balanced Accuracy	(TPR+TNR)/2	Mean per-class recall; treats rare attacks as equally important as normal.
MCC	$(TP\cdot TN-FP\cdot FN)/\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}$	Correlation over the confusion matrix in [−1,1]; **trustworthy under heavy imbalance**.
ROC-AUC	area under TPR vs FPR	Threshold-free ranking quality; **over-optimistic** when positives are rare.
PR-AUC (AP)	area under Precision-Recall	Honest ranking score for rare positives; baseline = prevalence.

Metrics deliberately excluded. Regression metrics (MAE/MSE/RMSE/R^2) are **not applicable** — this is classification, not regression. Raw **Accuracy is demoted** from a success metric to a diagnostic foil because, as shown, it is dominated by the majority class. We **lead with MCC, Balanced Accuracy, PR-AUC and per-class recall**, which remain informative under the 0.04%–13% class prevalences here.

Multiclass results on `KDDTest+`:

Model	Accuracy	Balanced Acc	Macro-F1	MCC
Baseline (majority)	0.4308	0.200	0.120	0.000
Logistic Regression	0.7397	0.5124	0.5178	0.6173
Random Forest	0.7531	0.4937	0.5083	0.6442
Gradient Boosting	**0.7791**	0.5626	0.5614	**0.6788**

Per-class recall on `KDDTest+` (the decisive table):

Class	LogReg	Random Forest	Gradient Boosting
normal	0.923	0.974	0.968
DoS	0.796	0.791	0.836
Probe	0.698	0.610	0.672
R2L	0.025	**0.050**	0.098
U2R	0.119	**0.045**	0.239

Balanced Accuracy (~0.49–0.56) is far below the 0.78 Accuracy — it is dragged down by the near-zero R2L/U2R recall that Accuracy hides. (It is still ~2.5× the 0.20 five-class chance floor, because `normal`/DoS/Probe are detected well; the failure is concentrated, not uniform.)

> **Sample-size caveat.** U2R has only **67** test rows, so its recall estimates (e.g. 0.045 ≈ 3/67) carry a wide confidence interval (95% CI ≈ ±5 pp by the normal approximation) and should be read as "almost entirely missed", not as precise point values. R2L (2,885 rows) is statistically solid.

Remediation attempt (cost-sensitive learning). Because the failure is concentrated in the rare classes, we retrained with `class_weight='balanced'`. The effect is **uneven and only partial**: Gradient Boosting's R2L recall improves from 0.098 to **0.277**, but Random Forest's R2L recall actually *drops* (0.050 → 0.016), and **both still miss the large majority of R2L/U2R**. This is strong evidence that the blindness is **not merely an imbalance artefact** — it stems from the train→test shift plus the fact that R2L/U2R are nearly inseparable from normal traffic at the flow-feature level. A real fix needs different telemetry (payload/host features) or a dedicated anomaly-detection paradigm, not just a heavier class weight — a conclusion the original tutorials never reach.

Testing our own recommendation: anomaly detection for the rare attacks. It would be hypocritical to recommend an anomaly-detection paradigm without testing it — the very omission we fault in the tutorials. So we trained two **semi-supervised one-class detectors on `normal` traffic only** (Isolation Forest; One-Class SVM, RBF) and scored them on `KDDTest+`, flagging deviations as attacks.

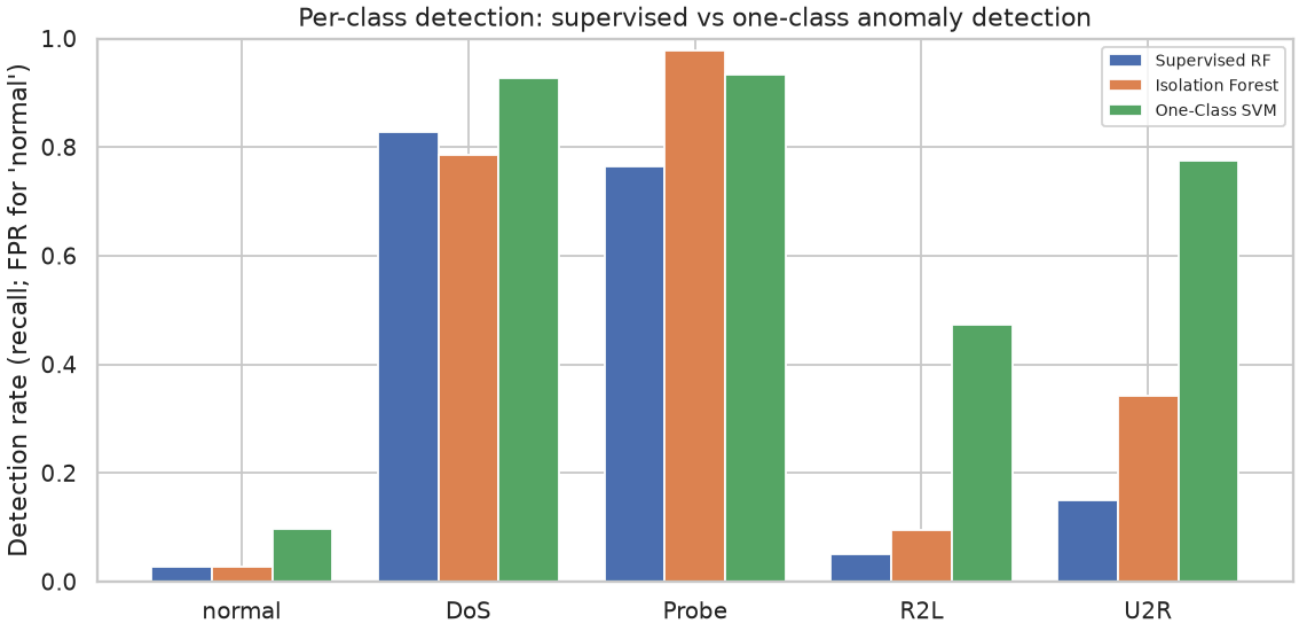
Model (`KDDTest+`)	Supervision	Recall	Precision	F2	MCC	ROC-AUC
Random Forest	supervised	0.6335	0.9689	0.6806	0.6214	0.9614
Gradient Boosting	supervised	0.6757	0.9693	0.7193	0.6552	0.9610
Isolation Forest	normal-only	0.6644	0.9696	0.7090	0.6466	0.9489

Model (`KDDTest+`)	Supervision	Recall	Precision	F2	MCC	ROC-AUC
One-Class SVM	**normal-only**	**0.8254**	0.9182	**0.8424**	**0.7214**	0.8832

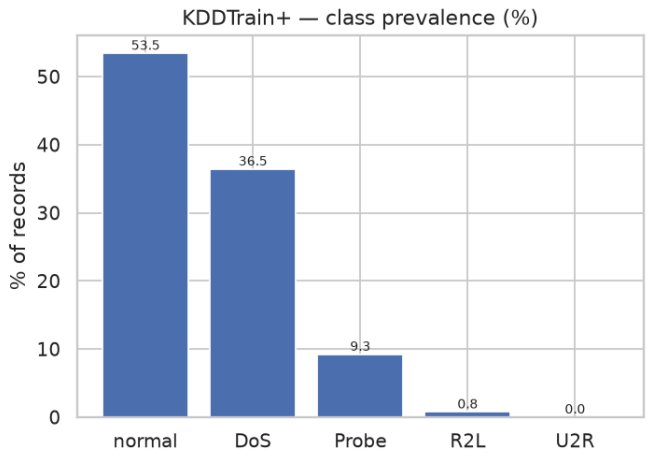
Per-family **detection rate** (fraction flagged as *attack*; the `normal` row is the false-positive rate):

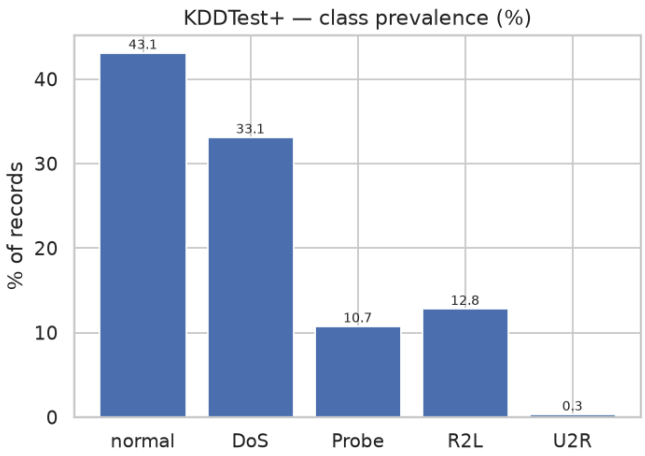
Class	Supervised RF	Isolation Forest	One-Class SVM
normal (FPR)	0.027	0.027	0.097
DoS	0.827	0.785	0.927
Probe	0.765	0.978	0.934
R2L	0.051	0.096	**0.473**
U2R	0.149	0.343	**0.776**

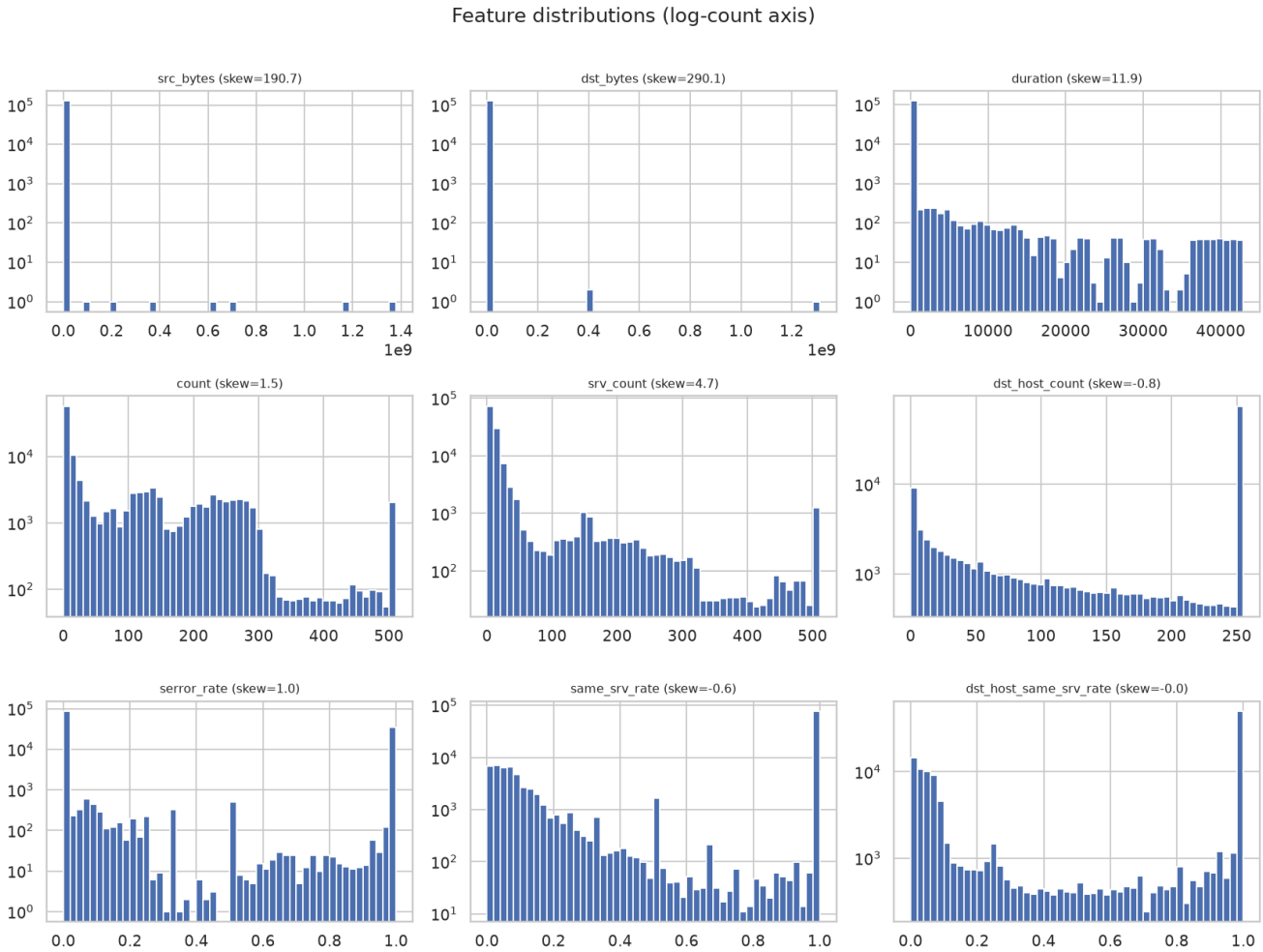
This is the most important positive result in the project. A One-Class SVM that *never sees a single attack label* **outperforms every supervised model on the official test set** (MCC 0.72 vs the best supervised 0.66; F2 0.84 vs 0.72) and, crucially, **recovers the rare attacks the supervised models miss**: R2L detection rises 0.05 → 0.47 and U2R 0.15 → 0.78. The mechanism is exactly our thesis — by modelling *only* what normal looks like, the detector is **immune to the train→test attack shift** that sinks the supervised classifiers. The cost is a higher false-alarm rate (normal FPR 0.03 → 0.10), i.e. the classic IDS precision/recall trade — but for a recall-critical task that is often the right operating point, and it is a trade a SOC can tune. **Anomaly detection is therefore not a free fix, but it is a demonstrably better paradigm for the highest-impact intrusions** — an evidence-backed conclusion, not the assertion the tutorials leave at "future work".

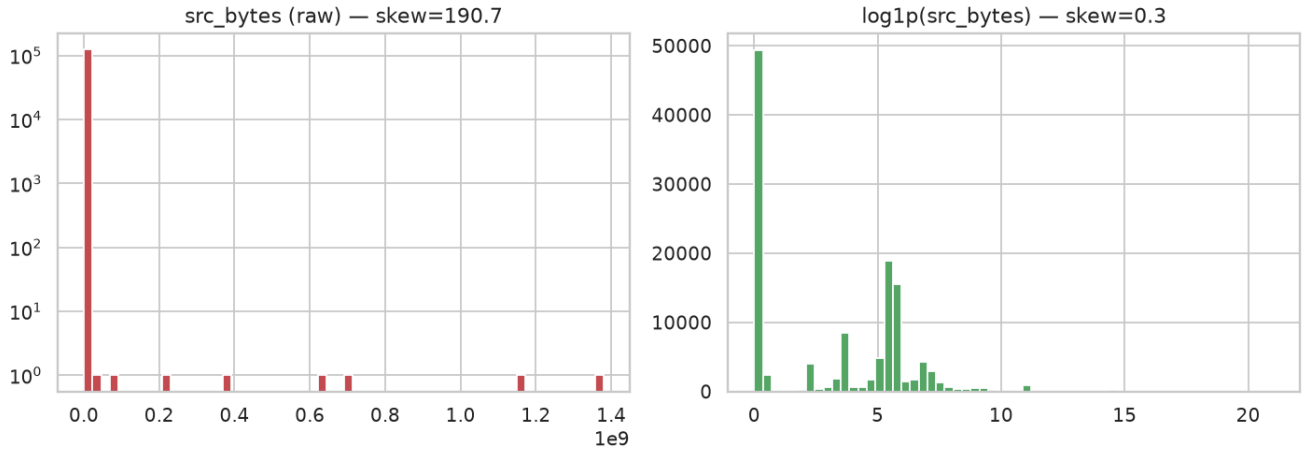


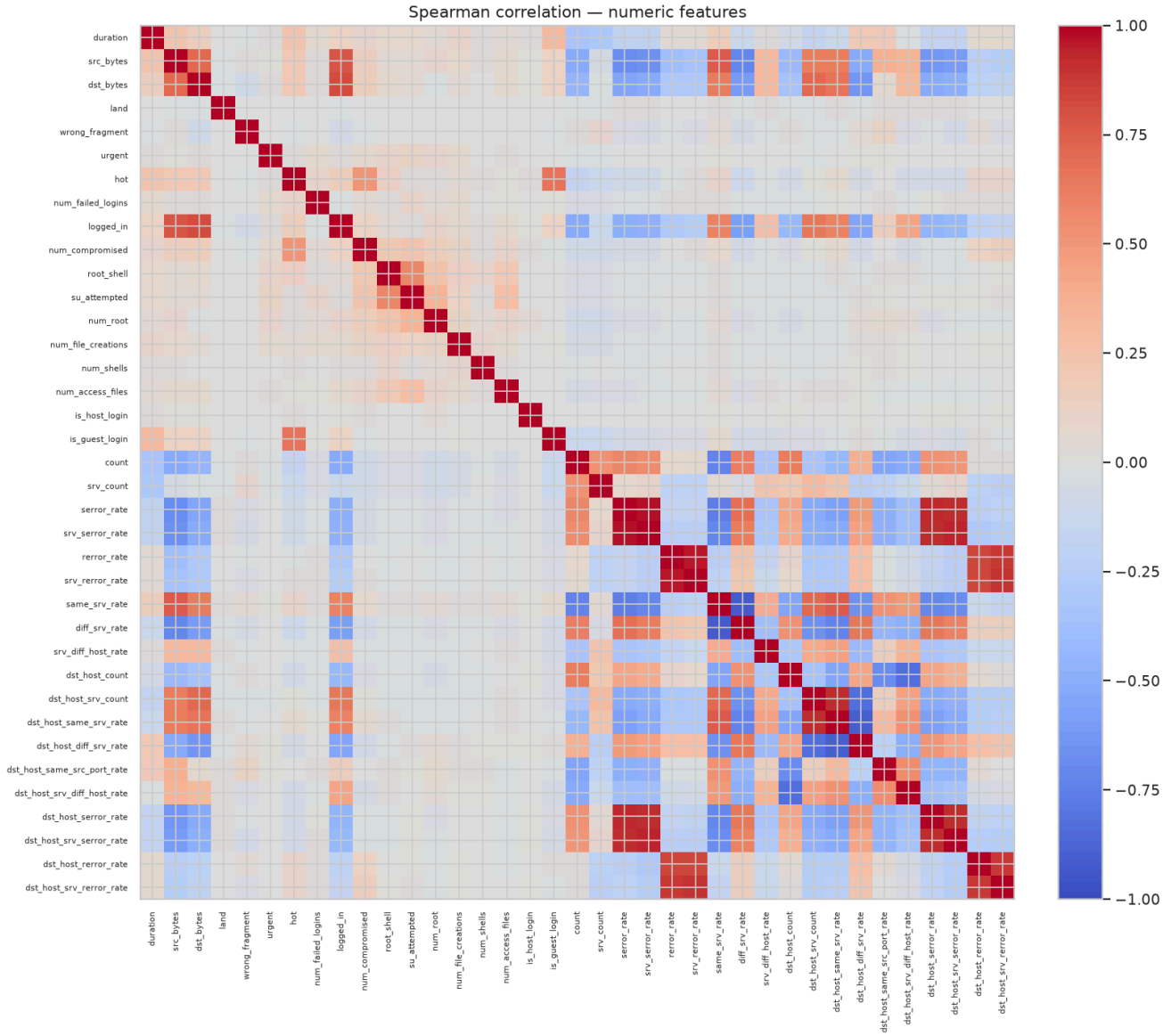
Figures: (1) class balance train vs test; (2) feature distributions; (3) log-transform effect; (4) Spearman heatmap; (5) PCA class separability; (6) PR & ROC curves on `KDDTest+`; (7) Random Forest confusion matrix; (8) per-class detection rate, supervised vs anomaly detection.

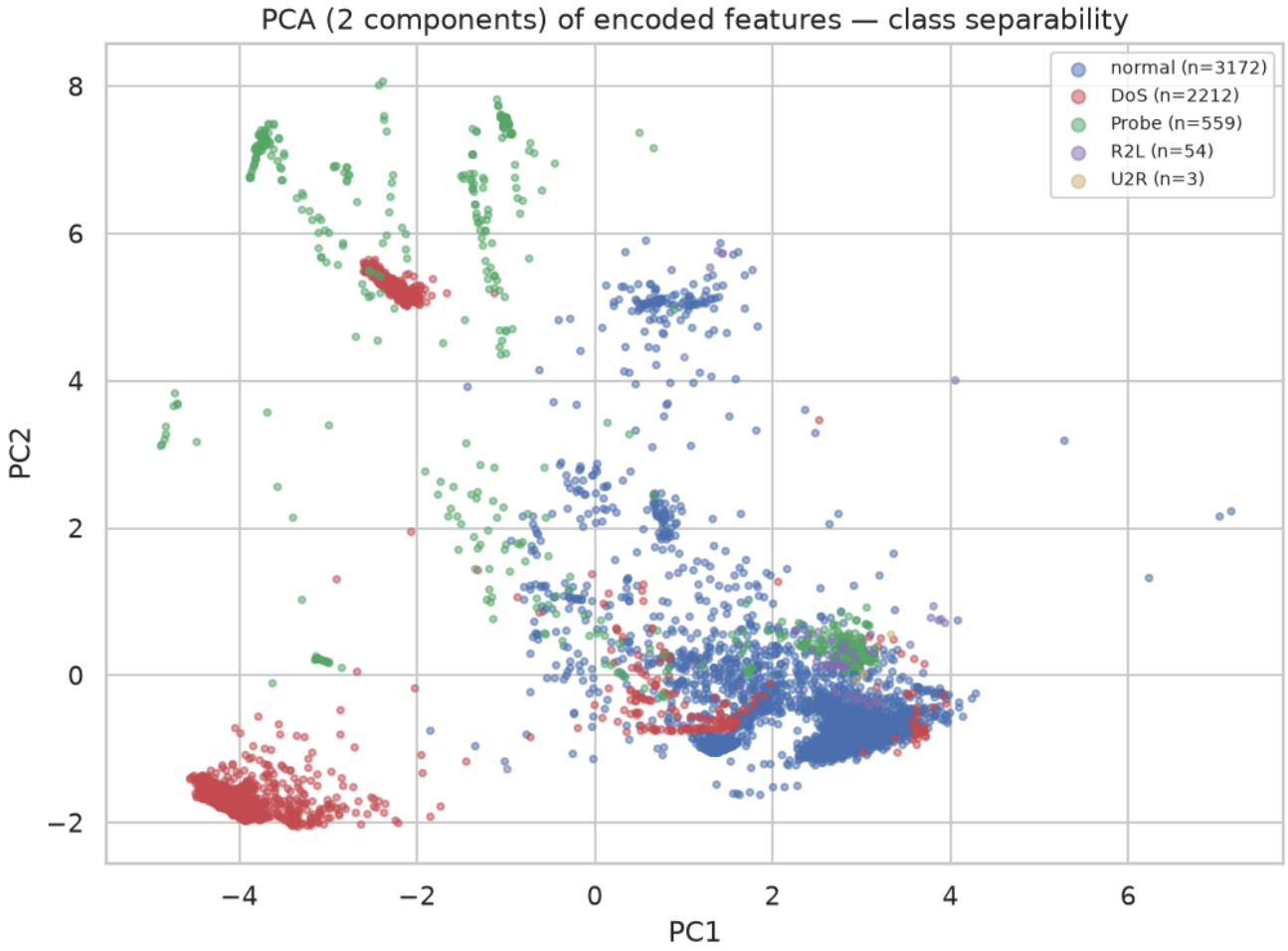

KDDTrain+ — class prevalence (%)
% of records
53.5
36.5
9.3
0.8
0.0
normal
DoS
Probe
R2L
U2R

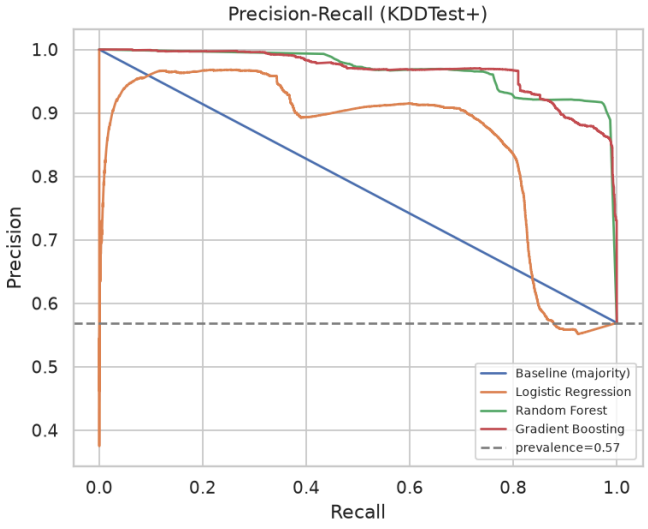

KDDTest+ — class prevalence (%)
% of records
43.1
33.1
10.7
12.8
0.3
normal
DoS
Probe
R2L
U2R

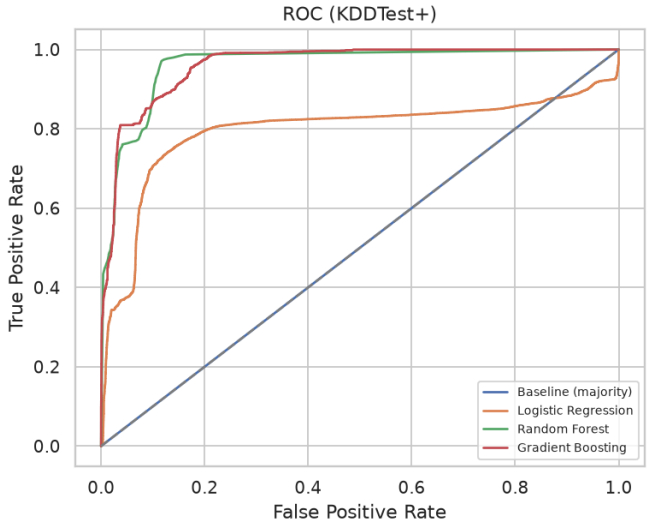

Feature distributions (log-count axis)
src_bytes (skew=190.7)
dst_bytes (skew=290.1)
duration (skew=11.9)
count (skew=1.5)
srv_count (skew=4.7)
dst_host_count (skew=-0.8)
serror_rate (skew=1.0)
same_srv_rate (skew=-0.6)
dst_host_same_srv_rate (skew=-0.0)

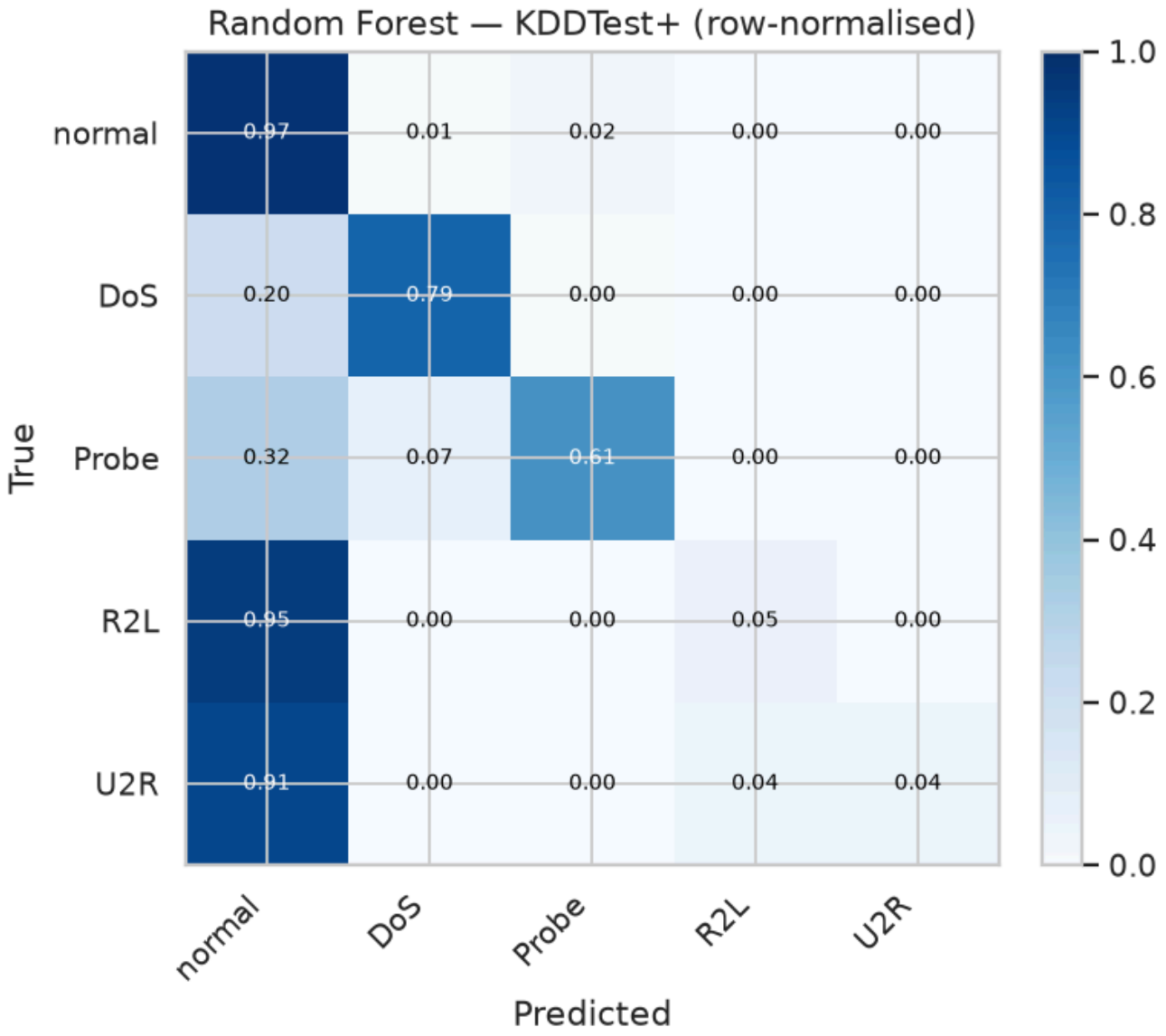

src_bytes (raw) — skew=190.7
log1p(src_bytes) — skew=0.3
1e9


Spearman correlation — numeric features
duration
src_bytes
dst_bytes
land
wrong_fragment
urgent
hot
num_failed_logins
logged_in
num_compromised
root_shell
su_attempted
num_root
num_file_creations
num_shells
num_access_files
is_host_login
is_guest_login
count
srv_count
serror_rate
srv_serror_rate
rerror_rate
srv_rerror_rate
same_srv_rate
diff_srv_rate
srv_diff_host_rate
dst_host_count
dst_host_srv_count
dst_host_same_srv_rate
dst_host_diff_srv_rate
dst_host_same_src_port_rate
dst_host_srv_diff_host_rate
dst_host_serror_rate
dst_host_srv_serror_rate
dst_host_rerror_rate
dst_host_srv_rerror_rate


PCA (2 components) of encoded features — class separability
normal (n=3172)
DoS (n=2212)
Probe (n=559)
R2L (n=54)
U2R (n=3)
PC1
PC2


Precision-Recall (KDDTest+)
Baseline (majority)
Logistic Regression
Random Forest
Gradient Boosting
prevalence=0.57
Precision
Recall


ROC (KDDTest+)
Baseline (majority)
Logistic Regression
Random Forest
Gradient Boosting
True Positive Rate
False Positive Rate



6. Error Analysis

Failure examples and patterns. Errors concentrate in **R2L** and **U2R**. The Random Forest confusion matrix (Fig. 5) shows R2L and U2R rows bleeding almost entirely into `normal`. This is the expected failure mode: R2L/U2R are **content/payload** attacks (guessed passwords, buffer overflows) that look statistically like normal connections at the flow level, **and** they are vanishingly rare in training (U2R = 52 of 125,973 rows = 0.04%) yet common in test (R2L = 12.8%).

Cybersecurity implications. The model is **operationally blind to the most damaging intrusions**. A defender trusting the 99% headline would be breached by exactly the credential-theft and privilege-escalation activity an IDS exists to catch.

The FP/FN trade-off. In IDS a missed attack (FN) usually costs more than a false alarm (FP), so we report **F2** (recall-weighted) and sweep the decision threshold:

Threshold	Precision	Recall	F2	False alarms (FP)	Missed attacks (FN)
0.10	0.921	0.852	0.865	935	1,893
0.50	0.969	0.637	0.684	261	4,655
0.90	0.970	0.522	0.575	206	6,132

Lowering the threshold from 0.5 to 0.1 recovers ~2,800 missed attacks at the cost of ~670 extra false alarms — a trade a real SOC must make deliberately. The default 0.5 threshold the tutorials use is **not** the right operating point for a recall-critical task.

But threshold tuning only slides along the supervised model's *mediocre* R2L/U2R curve. The larger lever is the **paradigm** (§5): the One-Class SVM recovers **47% of R2L and 78% of U2R** that *no* threshold on the supervised score can reach — because those attacks never cross the supervised decision boundary at all. The deepest error pattern, then, is not a threshold mis-set but a **model-class mismatch**: in-distribution supervised classification is the wrong tool for novel, content-based intrusions.

---

7. Conclusions

Key findings. 1. The headline "~99% accuracy" **reproduces** under the tutorials' protocol (random split of `KDDTrain+`: RF 0.9990; 5-fold CV 0.9989 ± 0.0002) but **does not survive** correct evaluation on the official `KDDTest+` (RF 0.7798 — a 21.9-point drop). 2. The cause is a **train→test distribution shift** (R2L 0.79% → 12.8%, U2R 0.04% → 0.30%) combined with **Accuracy on imbalanced data** — a trivial constant predictor already scores 0.569. 3. Under honest metrics the models **miss ~95% of R2L and U2R**, the highest-impact attacks; balanced accuracy (~0.49–0.56) sits far below the headline Accuracy because it refuses to let the well-detected `normal`/DoS/Probe classes paper over that collapse. 4. Cost-sensitive re-weighting helps only **partially and unevenly** — the failure is structural, not merely an imbalance artefact. 5. The data carries a constant feature (`num_outbound_cmds`), 9 redundant feature pairs, and a leakage trap (`difficulty`); pruning the redundancy costs nothing and three engineered features give a small, *tested* gain. 6. **The implied remedy works, and we proved it.** A one-class anomaly detector trained on normal traffic only **beats every supervised model on the official test set** (MCC 0.72 vs 0.66; F2 0.84 vs 0.72) and recovers the rare attacks (R2L 0.05 → 0.47, U2R 0.15 → 0.78) — because it is immune to the train→test attack shift. The cost is more false alarms; the *paradigm*, not a larger supervised model, is the lever that matters.

Lessons learned. In cybersecurity ML, the **evaluation protocol and metric choice decide the conclusion**. Reproducibility (getting the same number) is necessary but not sufficient — **validity** (measuring the right thing on the right distribution) is what matters. Accuracy without a baseline and without per-class recall is actively misleading on rare-attack problems.

Strengths of the proposed (original) solution. Simple, fast, and genuinely effective for the *common, high-volume* attacks (DoS/Probe recall 0.6–0.84); the feature set is rich and the dataset is clean and well-documented; the pipeline is easy to reproduce.

Weaknesses of the proposed solution. Evaluated on the wrong (random) split; reports only Accuracy; ignores class imbalance and per-class performance; blind to R2L/U2R; built on dated (1999-derived) traffic with no temporal validity; treats the problem as solved when it is not.

Suggestions for future improvements. (1) Always evaluate on the distribution-matched / temporally held-out test set; (2) report MCC, PR-AUC, Balanced Accuracy and **per-class recall**, never Accuracy alone; (3) **treat R2L/U2R as anomaly detection** — we showed a one-class SVM on normal-only traffic substantially outperforms the supervised models on the rare attacks (R2L 0.47, U2R 0.78 detection); pair it with cost-sensitive learning or host/payload telemetry to control its false-alarm rate; (4) tune the decision threshold to the SOC's FP/FN economics; (5) add richer features (inter-arrival Δt, per-host port entropy, byte asymmetry); (6) monitor for **concept drift** in production and retrain on current traffic.

---

8. Executive Summary

We critically reproduced the most common NSL-KDD intrusion-detection tutorial, which advertises **~99% accuracy**. By holding models, features, preprocessing and seeds fixed and changing **only** the evaluation protocol, we reproduced ~0.999 accuracy on a random split of `KDDTrain+` (and 0.9989 under 5-fold CV), then showed the identical Random Forest **falls to 0.78 accuracy on the official `KDDTest+`** — a 21.9-point drop. The drop is explained by a deliberate **train→test distribution shift** (R2L prevalence 0.79% → 12.8%, U2R 0.04% → 0.30%) and is invisible to Accuracy because of class imbalance: a constant predictor already scores 0.569. Under honest metrics (MCC 0.62, Balanced Accuracy 0.49, PR-AUC) and **per-class recall**, the models detect DoS/Probe but **miss ~95% of R2L and U2R** — the highest-impact attacks.

Methodology. We treated the project as a controlled experiment: faithful reproduction of the tutorial protocol, then a single change of variable (the test set) with everything else held fixed. We performed robust EDA (heavy-tail outliers via IQR/MAD, Spearman correlation, prevalence and a quantified train→test shift), built a leakage-safe scikit-learn pipeline (one-hot, log1p, scaling, constant-feature drop), trained four models (majority baseline, Logistic Regression, Random Forest, Gradient Boosting) with fixed seeds and cross-validation, and evaluated with an imbalance-aware metric suite plus per-class error analysis and a threshold sweep. We confirmed the gap is seed-invariant (five seeds), reproduced it with the tutorials' **own** leaky recipe (0.9996 → 0.8224), and probed two remedies: cost-sensitive re-weighting (helped only partially) and — decisively — a **semi-supervised anomaly-detection paradigm**. We also found a constant feature (`num_outbound_cmds`), nine redundant feature pairs (pruning them is free), and a leakage trap (`difficulty`).

Bottom line. The ~99% claim **is not supported** as a measure of real detection capability; it is an artefact of evaluation protocol and metric choice. We **do not recommend** adopting this approach unchanged: evaluate on the distribution-matched test set, report MCC/PR-AUC/per-class recall, and treat R2L/U2R as an **anomaly-detection** problem — which we did not merely recommend but **tested**: a one-class SVM trained on normal-only traffic beats the supervised models on the official test set (MCC 0.72 vs 0.66) and recovers the rare attacks (R2L 0.47, U2R 0.78), at a higher false-alarm rate. The repository contains a fully reproducible notebook, this PDF report, and all figures and metrics.

9. Summing It Up

- **Problem.** Detect network intrusions on NSL-KDD (binary and by attack family).
- **Selected source.** Popular NSL-KDD ML tutorials/repositories reporting ~99% accuracy via a random split of `KDDTrain+` (abhinav-bhardwaj; Mamcose), with the foundational dataset paper by Tavallaee et al. (2009).
- **Dataset.** NSL-KDD — `KDDTrain+` (125,973), `KDDTest+` (22,544), 41 features, 4 attack families.
- **Methodology.** Faithful reproduction (incl. a literal reconstruction of the tutorials' leaky recipe) + a controlled, multi-seed A/B protocol experiment; robust EDA; leakage-safe feature engineering with ablations; four supervised models **plus two anomaly detectors**; imbalance-aware metrics; error analysis. Fixed seeds; CV.
- **Main findings of the reproduction study.** The headline accuracy reproduces under the source's protocol (and under the tutorials' own pipeline) but **does not survive correct evaluation** (~0.99 → 0.78; gap 21.9 ± 0.0 pp over five seeds), and the supervised models fail precisely on the rare, dangerous attacks (R2L/U2R recall ≈ 0.05). **A one-class anomaly detector trained on normal traffic only reverses this** — beating the supervised models on the official test set (MCC 0.72 vs 0.66) and detecting 47%/78% of R2L/U2R — at a higher false-alarm rate.
- **Were the author's claims supported by our results? No.** The number is arithmetically correct but the *protocol* and *metric choice* make the claim misleading.
- **Most important insight.** In cybersecurity, **the evaluation protocol and the choice of metric decide the conclusion.** A model that looks 99% accurate can be operationally blind to the very attacks it must catch. "*All models are wrong, but some are useful*" — usefulness here means honest evaluation, not a high score.
- **Do we recommend using this project/approach on similar problems? Not as-is.** Recommended practice: evaluate on a distribution-matched / temporally-held-out test set; report MCC/PR-AUC/per-class recall instead of Accuracy; **use an anomaly-detection paradigm for the novel/content-based attacks** (we demonstrated a one-class SVM that outperforms supervised models on R2L/U2R), tuned via cost-sensitive learning and the decision threshold to the FP/FN economics of the SOC.
- **Final conclusion.** A rigorous, reproducible **refutation** of an over-optimistic but extremely common cybersecurity-ML claim — demonstrating that methodological rigor, not headline accuracy, is what makes a model trustworthy in cyber defence.

Reproducibility: all numbers above are emitted by `notebooks/analysis.ipynb` into `results/metrics.json`; figures are in `figures/`. Run `python notebooks/analysis.py` or execute the notebook to regenerate. `RANDOM_STATE = 42`.